Design Note

Julien Joly

18 March 2026

1 System Architecture

The engine is built using an asynchronous event-driven architecture via Python's `asyncio`. This setup ensures that data ingestion from multiple exchanges doesn't stall the pricing logic.

- **The Providers:** Three independent co-routines (Binance Spot, Coinbase Spot, and Binance Futures) handle the raw WebSocket streams. They capture Top-of-Book updates and push them into a central `asyncio.Queue`.
- **The Message Router:** A dedicated worker (`_process_messages`) pulls from the queue, identifies the source, updates the relevant `OrderBook` object, and triggers the Fair Value calculation.
- **The Heartbeat:** An `_output_loop` runs on a 500ms ticker. This acts as the "execution head," evaluating the current state, checking inventory risk, and generating quotes.

2 Fair-Value Methodology (Lead-Lag Model)

We don't just mid-price the spread. To avoid being picked off by informed traders, we use a simple predictive model combining spot liquidity and futures signals.

Step 1: Spot Micro-Price For each spot exchange, we calculate the micro-price to account for order book imbalance (I):

$$P_{micro} = \frac{P_{bid} \cdot Q_{ask} + P_{ask} \cdot Q_{bid}}{Q_{bid} + Q_{ask}}$$

where P is price and Q is quantity. This ensures that if the bid side is heavy, our fair value leans toward the ask.

Step 2: Consolidated Spot FV We weight the individual micro-prices by the relative liquidity (L) of each exchange:

$$FV_{spot} = \sum \left(P_{micro,i} \cdot \frac{L_i}{\sum L} \right)$$

Step 3: Futures Basis Adjustment Because Perpetual Futures often lead price discovery due to higher leverage, we blend the Binance perpertual price (P_{perp}) into our final value:

$$FV_{final} = (0.70 \cdot FV_{spot}) + (0.30 \cdot P_{perp})$$

3 Quote Logic & Inventory Risk

The primary goal is to remain delta neutral. We use a simple linear inventory skew to nudge the market into helping us rebalance.

- **Base Spread:** Orders are placed at $FV \pm (\text{Spread}/2)$.
- **The Skew:** If our position (q) is positive (long), we shift the entire quoting window down. This makes our ask more competitive (encouraging a sell) and our bid less attractive (discouraging a buy).

$$\text{Bid} = FV - \frac{S}{2} - (\lambda \cdot q) \quad , \quad \text{Ask} = FV + \frac{S}{2} - (\lambda \cdot q)$$

4 Failure Modes and Resiliency Protocol

The engine's reliability is governed by a degradation model. This mitigates the risk of trading in the dark (i.e. quoting based on stale, lagged or missing market data) by dynamically re-evaluating the integrity of its data sources on every 500ms.

Dynamic Re-weighting Mechanism The `FairValueCalculator` rejects static weighting in favor of a real-time staleness monitoring system:

- **Staleness Monitoring:** Every Best Bid/Offer update is assigned a `local_timestamp` immediately upon ingestion.
- **Threshold Enforcement:** If the latency gap between the current system time and the last update from a specific exchange exceeds the 2s staleness threshold, that exchange is flagged as unreliable.
- **Weight Redistribution:** Stale exchanges are assigned a weight of 0. The remaining healthy venues have their weights recalculated proportional to their relative liquidity (L):

$$W_{active,i} = \frac{L_i}{\sum L_{healthy}} \tag{1}$$

Operational Health States To balance market participation with capital preservation, the engine transitions between three distinct operational states:

State	Condition	Risk Response
NORMAL	2 Spot + Perp active	Full Lead-Lag Logic: Operates at baseline spread.
DEGRADED	1 Spot or Perp missing	Spread Expansion: Spreads widened.
SUSPENDED	0 Spot or Latency $> 2.0s$	Kill-Switch: Immediate withdrawal active limit orders.

Table 1: System Health States and Defensive Actions

Justification of Design Choices

1. **Liquidity-Proportional Weighting:** I chose to weight by total top-of-book volume ($BidSize + AskSize$) rather than an equal 50/50 split. This ensures that a single small-lot trade on a low-volume exchange (e.g., Coinbase) does not cause an artificial spike in Fair Value. The engine effectively follows the deepest market.

2. **The 2s Buffer:** This threshold was selected to absorb minor network jitters or WebSocket reconnections without triggering a full system halt. While high for institutional standards, it maintains stickiness in the market during non-critical volatility.

3. **Mandatory Spread Expansion:** When the engine loses a data source, it loses the ability to cross-verify prices. By widening the spread in the *DEGRADED* state, I ensure that any fills received during periods of uncertainty provide a higher profit margin to justify the increased risk of adverse selection.

5 Production Readiness

To move from this prototype to a production environment, the following changes are required:

1. **Language Migration:** Core paths should move to C++ to minimize microsecond-level jitter.

2. **OMS Implementation:** A robust Order Management System is needed to handle *Post-Only* orders and partial fills.

3. **Non-Linear Skew:** Implementing an exponential risk function (e^q) to defend the `max_position` more aggressively during high-volatility events.

4. **Fair Value and quote mechanics:** They are maybe too simple and not realistic. There exists much more sophisticated methods (works of Rosenbaum or Guéant are interesting) but there is a trade-off between optimal quoting strategies and computational efficiency.

5. **Multi-level quoting:** In my code, I provide one bid and one ask. In reality, a real market maker places a ladder of orders at different prices and sizes.

6 Results and Stress Tests

To validate the efficacy of the inventory management and skewed quoting logic, the engine was subjected to two controlled stress tests. In these scenarios, the system was initialized with a manual buy or sell order to create an immediate inventory imbalance of ± 2.5 BTC. These runs serve to demonstrate that the engine successfully detects non-neutral states and adjusts its quoting behavior to induce mean reversion, returning the book to a risk-neutral position while maintaining price discovery.

The plots can be found in the GitHub repository with the csv files.

Case Study 1: Initial Short Position (-2.5 BTC) In this run, the engine begins with a significant short bias. The expected behavior is an aggressive *Bid Skew* (buying above the mid-price) to attract sellers and neutralize the position.

The green area in the bottom chart shows a linear positive slope from -2.5 BTC toward 0.0 BTC over a 12-second window. The constant slope indicates that the InventoryManager effectively maintained a consistent bid skew. By pricing Bids aggressively, the engine attracted a steady stream of sell orders from the market, allowing it to buy back its short. The engine reached a neutral state (approx. -0.1 BTC) at the 12-second mark. Note how the slope flattens as it approaches zero; this demonstrates that the skew factor $\lambda \cdot q$ naturally decays as the risk decreases, preventing the engine from overshooting into a long position.

The top chart reveals the relationship between the three price feeds during a period of downward volatility. Between Seconds 7 and 9, the Binance Perp experienced two distinct step

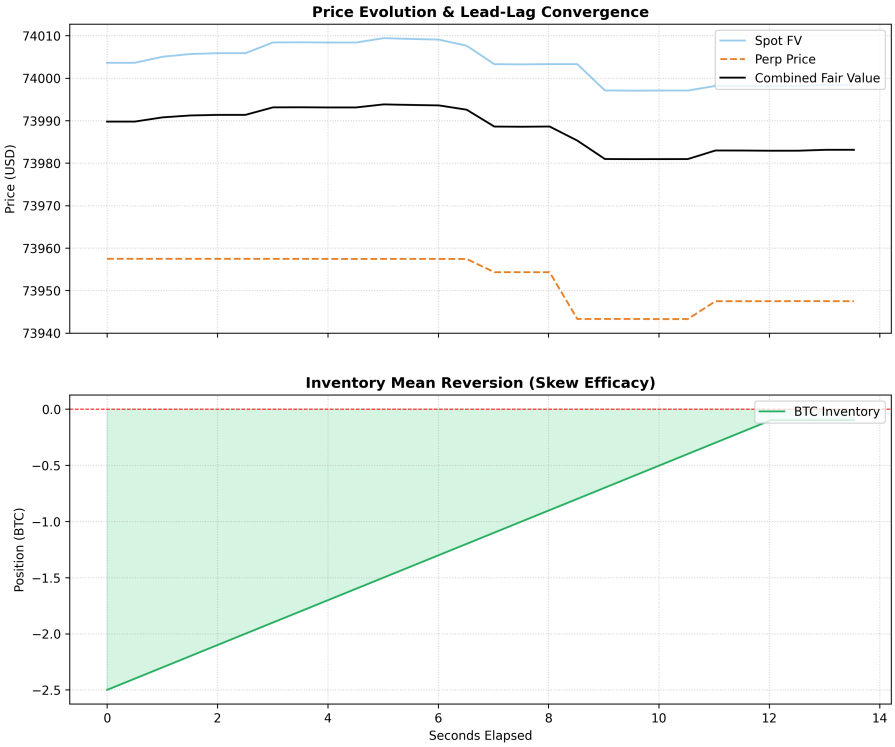

Figure 1: Evolution of a -2.5 BTC Short Position. The bottom pane illustrates a consistent positive slope as the engine successfully executes buy-side fills to reach neutrality.

drops. The fair value reacted almost instantly to these futures moves. The spot FV lagged significantly, staying at higher levels for several heartbeats before finally adjusting downward. By leaning on the perp lead, the engine avoided buying back its short at the stale (higher) spot price. This saved the strategy approximately \$10 to \$15 per BTC in execution costs by not chasing a price that was already declining in the futures market.

Status	Position (q)	Fair Value	Bid (Buy)	Ask (Sell)
NORMAL	-2.3	73990.75	74001.25	74003.25
NORMAL	-2.2	73991.20	74001.20	74003.20
NORMAL	-0.2	73982.95	73982.95	73984.95
NORMAL	-0.1	73982.90	73982.40	73984.40

Table 2: Short position trace: Quote prices are skewed upward to attract liquidity.

Looking at the loggs of the run (Table 3), we can see the internal logic that drove the plot: when we had a -2.5 position, the fair value was 73990.75, but the bid was set way higher at 74001.25. Usually, a bid is set below fair value (to buy cheap). However, because the inventory was -2.5, the engine purposely set its bid above fair value. This mechanic ensures the engine gets the fills it needs to close its risky short position quickly, even if it has to pay a small premium to the market.

When we have less inventory (say -0.1), the quotes are much closer to the fair value (see the last two lines of the table).

Case Study 2: Initial Long Position (+2.5 BTC) This scenario tests the ask skew logic. The engine must lower its selling price relative to the Fair Value to incentivize buyers to take its inventory.

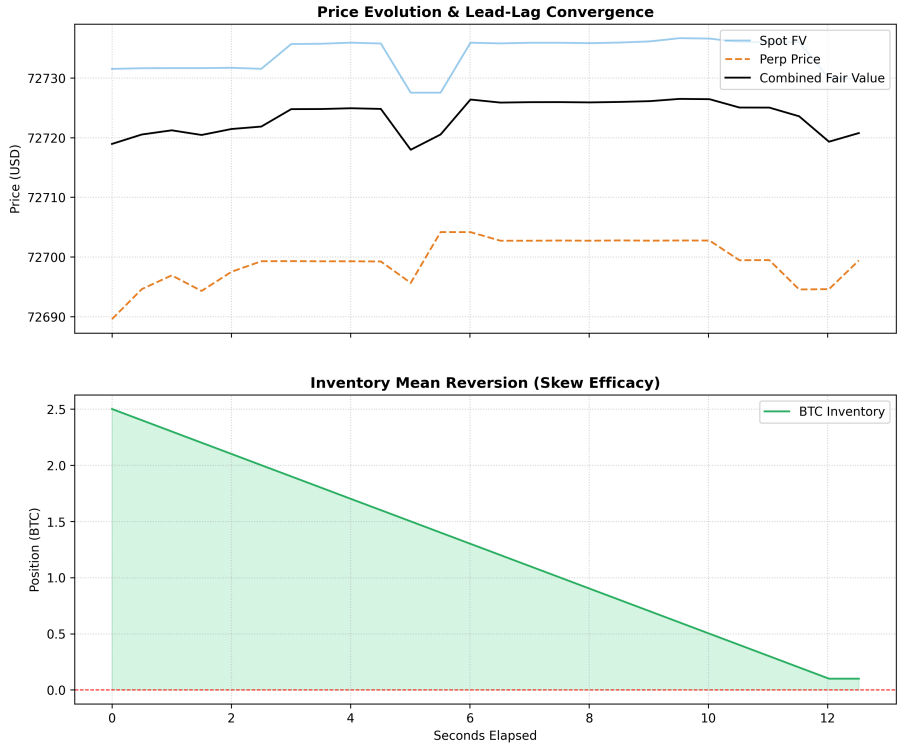

Figure 2: Evolution of a +2.5 BTC Long Position. The downward slope in the inventory pane confirms the efficacy of the ask-side skewing logic under long pressure.

We have a similar but reversed behavior for the initial long position.

Status	Position (q)	Fair Value	Bid (Buy)	Ask (Sell)
NORMAL	1.9	72724.77	72714.27	72716.27
NORMAL	1.8	72724.79	72714.79	72716.79
NORMAL	0.2	72723.57	72721.57	72723.57
NORMAL	0.1	72719.31	72717.81	72719.81

Table 3: Long position trace: Quote prices are skewed downward to attract liquidity.

7 Some Hyper-Parameters

Parameter	Value	Function
`perp_weight`	0.3	Influence of Futures on Fair Value.
`base_spread`	\$2	Minimum width to capture alpha/fees.
`max_position`	10 BTC	Hard risk limit for the engine.
`staleness_threshold`	2s	Timeout before an exchange is ignored.

8 Conclusion

The results of this prototype demonstrate that the core mechanics of a high-frequency market-making engine (namely multi-venue price discovery and inventory-responsive skewing) are functioning as intended. The engine successfully identifies Lead-Lag relationships and maintains inventory neutrality through dynamic quoting, as evidenced by the empirical results.

However, it is critical to acknowledge that this is a simplified foundational setup. While the logic is sound, several high-priority hurdles must be addressed before this architecture could be considered for live capital deployment:

- **Execution Latency:** The current Python-based `asyncio` loop, while efficient for prototyping, would likely face disadvantages against C++ based competitors in a true HFT environment.
- **Toxic Flow Detection:** The engine currently treats all counterparties equally. In live markets, a more robust system would require OFI metrics to avoid being picked off by informed traders.
- **Fee Structure:** Real-world profitability is highly sensitive to exchange maker/taker rebates, which are not currently modeled in this spread calculation.